Le Mans Université
Licence Informatique *2ème année*
Module 174UP02 Rapport de Projet

Codenames

PIAU Romain - ROGER Noam
QUINTON Chloé - MAUDET Mathis

8 avril 2026

Table des matières

1 Introduction **3**

2 Conception **3**
2.1 Présentation du jeu 3
2.2 Notre adaptation numérique 5

3 Organisation du travail **6**

4 Développement **7**
4.1 Structure du code 7
4.2 Principales structures de données 8
4.3 Interface graphique 8
4.4 Gestion des parties 8
4.5 Fonctionnalités additionnelles 8

5 Bilan et conclusion **9**

6 Annexes **10**

1 Introduction

Ce document présente le projet Codenames, développé dans le cadre du projet de 2ème année de licence informatique à l'Université du Mans. Le projet consiste en la création d'un jeu numérique basé sur le concept du jeu de société Codenames. Les fonctionnalités principales incluent la gestion des parties, l'interface utilisateur et l'intégration des règles du jeu. Ce rapport détaille les choix techniques, l'architecture du système et les résultats obtenus lors des tests.

2 Conception

2.1 Présentation du jeu

Codenames est un jeu d'association d'idées en équipes. Chaque équipe doit retrouver ses mots secrets à partir d'indices donnés par son maître-espion.

Objectif : être la première équipe à retrouver tous ses agents.

1. Mise en place :
   - Former 2 équipes : rouge et bleue
   - Choisir un maître-espion par équipe
   - Disposer 25 mots en grille (5x5)
   - Tirer une carte clé au hasard

     Important : La carte clé indique :
     - 9 mots pour une équipe (celle qui commence)
     - 8 mots pour l'autre
     - 7 civils
     - 1 assassin
2. Déroulement d'un tour

   (a) Donner un indice

   Le maître-espion doit dire :
   - Un mot unique
   - Un nombre

Exemple :

Indice : **Animal 2**

Cela signifie que 2 mots sur la table sont liés à "animal".

Règles strictes pour les indices

- Un seul mot (pas de phrase)
- Pas un mot présent sur la table
- Pas une partie d'un mot (ex : "auto" pour "automobile" interdit)
- Pas de traduction directe (ex : dire "dog" pour "chien")
- Pas de communication non verbale

(b) Phase de devinettes

Les joueurs peuvent proposer des mots.

- Nombre maximum de propositions = nombre annoncé + 1

Exemple :

Indice : **Animal 2**

→ L'équipe peut proposer jusqu'à 3 mots.

Résultat des choix

- Bon mot
  - On recouvre avec la couleur de l'équipe
  - L'équipe peut continuer
- Mot neutre
  - Tour terminé immédiatement
- Mot adverse
  - Tour terminé
  - Aide l'équipe adverse
- Assassin

  Effet : défaite immédiate

3. Fin de partie
   - Une équipe gagne si elle trouve tous ses mots
   - Une équipe perd immédiatement si elle choisit l'assassin
4. Règles avancées

Indice avec 0

Exemple : **Animal 0**

Signifie : Aucun mot restant n'est lié à "animal".
→ Permet d'éviter certains pièges.

2.2 Notre adaptation numérique

Notre version numérique de Codenames vise à recréer l'expérience du jeu de société tout en offrant des fonctionnalités supplémentaires pour améliorer l'interaction et la convivialité.

Fonctionnalités principales :

1. Interface utilisateur intuitive grâce à SDL
   — différentes intefaces créées pour chaque étape (menu, lobby, jeu),
   — destion avancée de bouttons et inputs,
   — bandeau d'informations (affichage fps, réglage du volume et de la luminosité, règles du jeu)...
2. Gestion automatique des règles du jeu, le code permet
   — de vérifier la validité des indices donnés par les maîtres-espions,
   — de gérer les différentes phases du jeu (donner un indice, devinettes, fin de partie),
   — d'appliquer les conséquences des choix des joueurs (bon mot, mot neutre, mot adverse, assassin).
3. Possibilité de jouer en ligne avec des amis grâce à la création de lobbies privés
4. Système de chat intégré pour communiquer entre les joueurs
5. Historique des indices

3 Organisation du travail

Tâches	**Noam**	**Romain**	**Chloé**	**Mathis**
Conception de la structure du code	40%	30%	15%	15%
Fonctions principales (serveur)	30%	30%	20%	20%
Fonctions principales (client)	20%	20%	30%	30%
Affichage de l'interface graphique	10%	30%	30%	30%
Gestion du lobby	20%	20%	15%	45%
Gestion des fonctions du jeu	25%	25%	25%	25%
Rédaction du rapport de projet	15%	15%	55%	15%
Tests et débogage	25%	25%	25%	25%

Table 1 – Répartition des tâches entre les membres de l'équipe

4 Développement

4.1 Structure du code

Le code du jeu est structuré de la manière suivante :
- Deux parties principales : client et serveur
- Un dossier de tests
- Un dossier de documentation
- Un dossier rapport
- Un fichier README.md pour expliquer les principales fonctionnalités du jeu et les instructions d'installation et d'utilisation
- Un fichier TODO.md pour suivre les tâches à accomplir et les fonctionnalités à implémenter
- Un fichier .gitignore pour exclure les fichiers temporaires et les ressources non nécessaires du dépôt Git
- Un fichier LICENSE pour spécifier les conditions de distribution et d'utilisation du code
- Un fichier setup pour faciliter l'installation des dépendances et la configuration de l'environnement de développement
- Un fichier doxyfile pour la génération de la documentation avec Doxygen

Les parties client et serveur sont les plus développées et contiennent les fonctionnalités principales du jeu. Elles partagent une structure similaire pour faciliter la maintenance et la compréhension du code.

1. **Points communs des deux parties**
   - Dossier assets : contient les ressources graphiques, textuelles ou sonores du jeu
   - Dossier lib : contient les fichiers d'en-tête et les bibliothèques
   - Dossier src : contient le code source
   - Fichier Makefile : pour la compilation du projet
   - Fichier run : pour lancer le jeu
2. **Côté client**
   - Bibliothèques SDL compilés (ajouté par le script setup)
   - Gestion de l'interface graphique (menu, lobby, jeu)
   - Traitement des bouttons, inputs utilisateur, crossfade, bandeau d'informations...
   - Communication avec le serveur (envoi des actions du joueur, réception des mises à jour du jeu)
3. **Côté serveur**
   - Gestion des connexions clients
   - Implémentation de la logique du jeu

4.2 Principales structures de données

- **AppContext** : contient les informations sur l'état de l'application. Utilité : centraliser les données et les ressources du jeu pour faciliter la gestion de l'état et de l'affichage. Détails :
  - **Gérer l'affichage SDL**
    - `window` → la fenêtre du jeu
    - `renderer` → le moteur de rendu graphique
    - `bg_color` → couleur de fond
  - **Gérer l'état du jeu**
    - `app_state` → menu, lobby, en partie, etc.
    - `lobby` → données du lobby (joueurs, salle, etc.)
  - **Gérer le joueur**
    - `player_role` → agent ou spy
    - `player_team` → équipe
    - `player_id` → identifiant serveur
    - `player_uuid`, `player_name` → identité persistée
  - **Gérer la connexion réseau**
    - `sock` → socket réseau
    - `ping_ms` → latence avec le serveur
  - **Gérer le timing et les performances**
    - `clock` → nombre de frames écoulées
    - `fps` → images par seconde
    - `frame_start_time` → début de frame
    - `last_fullscreen_toggle_ms` → évite les toggles rapides
  - **Gérer l'audio**
    - `music_volume`
    - `sound_effects_volume`
  - **Gérer la fenêtre**
    - taille et position en mode fenêtré (`windowed_width`, `windowed_height`, etc.)
  - **Informations générales**
    - `version` → version du jeu

4.3 Interface graphique

4.4 Gestion des parties

4.5 Fonctionnalités additionnelles

5 Bilan et conclusion

6 Annexes